Белорусский государственный университет
Факультет прикладной математики и информатики
Кафедра технологий программирования

Отчёт по лабораторной работе № 4

по курсу «Проектирование человеко-машинных интерфейсов»

Стили интеграции программных систем и проектирование API

Проект: МаркетШоп.by — сервис автоматической генерации описаний товаров в рекомендательной системе маркетплейса одежды и обуви (Республика Беларусь)

Выполнили: студенты 4 курса, группа 10
Харченко Анастасия
Шепко Варвара
Степук Анастасия

Проверила: старший преподаватель кафедры ТП
Давидовская М. И.

Минск, 2026

Содержание

Введение 3
1 Модель требований и реестр вариантов использования 4
1.1 Анализ требований ЛР№3 4
1.2 Действующие лица 5
1.3 Модель требований 5
1.4 Реестр вариантов использования 8
2 Физическая модель базы данных 10
2.1 Уточнение логической модели ЛР№3 10
2.2 Логическая схема 10
2.3 Физическая модель 11
2.4 Проверка схемы 17
3 Маршруты и конечные точки API 18
3.1 Принципы проектирования 18
3.2 Маршруты 18
3.3 Варианты использования и конечные точки 20
4 Проектирование и тестирование API 23
4.1 Спецификация OpenAPI 23
4.2 Тела POST/PUT-запросов и HTTP-статусы 24
4.3 Mock-сервер и коллекция запросов 27
4.4 Результаты тестирования 28
5 MCP-сервер 29
5.1 Назначение и анализ примеров 29
5.2 Архитектура 29
5.3 Tools, Resources и Prompts 31
5.4 Реализация 32
5.5 Тестирование 33
5.6 Документация MCP-сервера 35
6 Документирование проекта 36
Заключение 37
Список использованных источников 38
Приложение А. Логическая схема базы данных 39
Приложение Б. Диаграмма файлов проекта 47

Введение

Лабораторная работа продолжает проект МаркетШоп.by из ЛР № 1–ЛР № 3. В ЛР № 3 были разработаны 20 пользовательских историй, диаграммы вариантов использования, классов и ERD, модель C4 и техническое задание по шаблону Вигерса. В этой работе требования доведены до уровня интерфейсов серверной части: спроектированы REST API и MCP-сервер, через который AI-ассистент работает с данными маркетплейса.

Для группы 10 выполнены задания 1–5. Задание 6 (инфраструктура, Docker, CI/CD) к группе 10 не относится и не выполнялось.

Все артефакты сохранены в репозитории проекта и связаны между собой: таблицы требований, логическая схема БД, таблицы маршрутов и ER-диаграммы не набирались вручную, а сформированы скриптами из трёх исходных файлов — `requirements/requirements.yaml`, `db/schema.sql` и `api/openapi.yaml`. Скрипт `tools/gen_api_docs.py` одновременно проверяет согласованность: каждая User Story ссылается на существующий вариант использования, каждый вариант использования реализован маршрутом API или инструментом MCP, каждый маршрут сопоставлен таблицам БД. Последний запуск проверки ошибок не выявил.

Таблица 1 — Результаты работы по заданиям

Задание	Результат	Файлы
1. Модель требований и реестр UC	32 User Story (12 новых), 30 вариантов использования (10 новых), 9 действующих лиц	`requirements/requirements.yaml`, `wiki/Функциональные-требования.md`
2. Физическая модель БД	логическая схема 31 таблицы, 4 ER-диаграммы, SQL для PostgreSQL 16, проверка в реальной СУБД	`db/schema.sql`, `db/seed.sql`, `db/verify_schema.py`, `wiki/Схема-базы-данных.md`
3. Маршруты и конечные точки	45 маршрутов по 23 сущностям, сопоставление 55 операций с UC	`wiki/API-проекта.md`
4. Проектирование и тестирование API	спецификация OpenAPI 3.0.3, mock-сервер, коллекция Bruno из 43 запросов, Swagger UI	`api/openapi.yaml`, `api/bruno/`, `api/results/`, `docs/api/`
5. MCP-сервер	MVP на MCP Python SDK: 13 Tools, 8 Resources, 3 Prompts, SQLite, 23 автотеста, проверка MCP Inspector	`mcp-server/`
Документирование	7 страниц вики, сайт GitHub Pages, README	`wiki/`, `docs/`, `README.md`

1 Модель требований и реестр вариантов использования

1.1 Анализ требований ЛР№3

Требования ЛР № 3 описывали поведение интерфейса: что видит покупатель в карточке, как модератор работает с очередью. Для API нужен другой срез — какие операции над какими данными доступны каждому действующему лицу. При сопоставлении историй ЛР№3 с будущими маршрутами выявлено три вида пробелов.

1. **Варианты использования без историй.** На диаграмме ЛР № 3 были UC-05 «Оформить заказ», UC-06 «Оплатить заказ», UC-07 «Отследить доставку», UC-09 «Оставить отзыв», UC-15 «Обработать заявку на возврат», UC-20 «Авторизоваться», но ни одна User Story их не обосновывала: истории ЛР№3 были сосредоточены на описаниях товаров и возвратах. Добавлены US-21… US-24 и US-27.
2. **Операции, без которых API неполон.** Регистрация, отмена заказа, удаление учётной записи (ФТ-13.5 из ТЗ ЛР № 3), управление вариантами и остатками (ФТ-11.5), приём вебхука платёжного шлюза (ФТ-15.3, ФТ-15.6), журнал действий (ФТ-13.6). Для них добавлены US-25, US-26, US-28, US-29 и UC-21, UC-22, UC-25, UC-28, UC-29.
3. **Истории, которым нужен отдельный UC.** US-12 «Условия возврата видны до оформления заказа» в ЛР № 3 входила в UC-03 «Просмотреть карточку товара». Для API условия возврата — самостоятельный ресурс, который читают и карточка, и экран оформления заказа (АС-12.3 требует единого источника правил), поэтому выделен UC-23. По той же причине выделены UC-24 (партия товаров, US-16), UC-26 (аналитика, US-17) и UC-27 (обязательные атрибуты, US-18).

Ещё два решения изменили набор действующих лиц. Роль «Администратор-модератор» ЛР № 3 разделена на **Модератора** и **Администратора**: в API это разные роли JWT, модератор принимает решения по карточкам, но не меняет правила генерации. Для задания 5 добавлено действующее лицо **AI-ассистент** и UC-30 «Выполнить операцию через AI-ассистента» с историями US-30…US-32.

История US-20 «Подписка на поступление размера» (приоритет Low) оставлена в модели с пометкой «отложено»: в первую версию API она не входит.

Нумерация ЛР № 3 (US-01…US-20, UC-01…UC-20) сохранена без изменений, чтобы трассировка «опрос → персонаж → история → маршрут» не прерывалась.

1.2 Действующие лица

Таблица 2 — Действующие лица и их представление в API

Действующее лицо	Тип	Представление в API
Гость	человек	публичные операции без токена
Покупатель	человек	JWT, роль `buyer`
Продавец	человек	JWT, роль `seller`
Модератор	человек	JWT, роль `moderator`
Администратор	человек	JWT, роль `admin`
Система генерации описаний	внутренняя система	вызывается сервером при сохранении атрибутов и правке текста
Платёжный шлюз	внешняя система	вебхук `POST /payments/webhook` с HMAC-подписью
Служба доставки	внешняя система	источник трек-номера в ответе `GET /orders/{orderId}`
AI-ассистент	внешняя система	протокол MCP, MCP-сервер `marketshop-assistant`

1.3 Модель требований

Модель представлена в форме «Действующее лицо → User Story → Use Case» (таблица 3). Истории, добавленные в ЛР№4, отмечены индексом «ЛР4».

Таблица 3 — Модель требований «Действующее лицо → User Story → Use Case»

Действующее лицо	User Story	Use Case	Приоритет
Покупатель	**US-01** Как покупатель, я хочу видеть на главной подборку товаров под мой размер, стиль и сезон, чтобы не перебирать каталог вручную	UC-01 Просмотреть ленту рекомендаций	High
Покупатель	**US-02** Как покупатель, я хочу видеть в карточке описание с составом, посадкой и уходом, чтобы решить о покупке без вопроса продавцу	UC-03 Просмотреть карточку товара	High
Покупатель	**US-03** Как покупатель, я хочу получить рекомендацию размера с пояснением о посадке, чтобы заказать нужный размер с первого раза	UC-04 Подобрать размер	High
Покупатель	**US-04** Как покупатель, я хочу ввести параметры размера прямо в карточке, чтобы не уходить в настройки профиля	UC-04 Подобрать размер	Medium
Продавец	**US-05** Как продавец, я хочу получить черновик описания по заполненным атрибутам, чтобы опубликовать коллекцию за один заход	UC-10 Загрузить товар и заполнить атрибуты UC-11 Сформировать черновик описания	High
Продавец	**US-06** Как продавец, я хочу видеть незаполненные обязательные атрибуты до генерации, чтобы не публиковать неполное описание	UC-12 Проверить полноту атрибутов	High
Продавец	**US-07** Как продавец, я хочу видеть подсказки о посадке из отзывов рядом с текстом, чтобы уточнить описание	UC-13 Отредактировать описание	Medium

Действующее лицо	User Story	Use Case	Приоритет
Модератор	**US-08** Как модератор, я хочу видеть очередь, отсортированную по уверенности генератора, чтобы сначала проверять рискованные тексты	UC-14 Отправить карточку на модерацию UC-16 Проверить карточку в очереди модерации	High
Модератор	**US-09** Как модератор, я хочу, чтобы система блокировала неподтверждённые заявления о качестве, чтобы продавец не маскировал плохой товар	UC-17 Заблокировать запрещённые формулировки UC-18 Настроить правила генерации	High
Модератор	**US-10** Как модератор, я хочу выбрать готовый шаблон комментария при возврате карточки, чтобы не набирать типовые пояснения	UC-16 Проверить карточку в очереди модерации UC-18 Настроить правила генерации	Low
Покупатель	**US-11** Как покупатель, я хочу оформить возврат или обмен за два-три шага, чтобы не обращаться в поддержку	UC-08 Оформить возврат или обмен	High
Покупатель	**US-12** Как покупатель, я хочу видеть условия возврата в карточке и при оформлении заказа, чтобы покупать без опасений	UC-23 Ознакомиться с условиями возврата	Medium
Покупатель	**US-13** Как покупатель, я хочу отфильтровать товары «Сделано в Беларуси», чтобы поддержать местные бренды	UC-02 Найти товар	Medium
Покупатель	**US-14** Как покупатель, я хочу найти товар по штрихкоду или фотографии, чтобы не вводить запрос вручную	UC-02 Найти товар	Medium
Покупатель	**US-15** Как покупатель, я хочу видеть понятную причину, когда возврат недоступен, чтобы знать, что делать дальше	UC-08 Оформить возврат или обмен	High
Продавец	**US-16** Как продавец, я хочу загрузить партию товаров и видеть статус каждой позиции, чтобы неполные позиции не тормозили остальные	UC-24 Загрузить партию товаров	Medium
Продавец	**US-17** Как продавец, я хочу видеть влияние описаний на конверсию и возвраты, чтобы улучшать карточки	UC-26 Просмотреть аналитику описаний	Medium
Администратор	**US-18** Как администратор, я хочу настраивать обязательные атрибуты без выпуска версии, чтобы не зависеть от релизного цикла	UC-27 Настроить обязательные атрибуты	High
Администратор	**US-19** Как администратор, я хочу видеть рейтинг качества карточек продавца и повторные нарушения, чтобы выявлять недобросовестных продавцов	UC-19 Контролировать повторные нарушения продавца	Medium
Покупатель	**US-20** Как покупатель, я хочу подписаться на поступление нужного размера, чтобы не проверять наличие вручную *(отложено, вне API v1)*	UC-03 Просмотреть карточку товара	Low
Гость	**US-21** ЛР4 Как гость, я хочу зарегистрироваться и входить по e-mail и паролю, чтобы оформлять заказы и сохранять параметры размера	UC-21 Зарегистрироваться UC-20 Авторизоваться	High

Действующее лицо	User Story	Use Case	Приоритет
Покупатель	**US-22** ЛР4 Как покупатель, я хочу оформить заказ и оплатить его картой или через ЕРИП, чтобы получить товар в удобном пункте выдачи	UC-05 Оформить заказ UC-06 Оплатить заказ	High
Покупатель	**US-23** ЛР4 Как покупатель, я хочу видеть статус и трек-номер заказа и отменить его до отправки, чтобы управлять покупкой без звонка в поддержку	UC-07 Отследить доставку UC-29 Отменить заказ	Medium
Покупатель	**US-24** ЛР4 Как покупатель, я хочу оставить отзыв с оценкой посадки, чтобы помочь другим выбрать размер	UC-09 Оставить отзыв с оценкой посадки	Medium
Покупатель	**US-25** ЛР4 Как покупатель, я хочу удалить учётную запись, чтобы мои персональные данные были обезличены (ФТ-13.5)	UC-22 Удалить учётную запись	High
Продавец	**US-26** ЛР4 Как продавец, я хочу задавать варианты товара по размеру и цвету и остатки по складам, чтобы покупатель видел реальное наличие	UC-25 Управлять вариантами и остатками	High
Продавец	**US-27** ЛР4 Как продавец, я хочу рассматривать заявки на возврат по своим товарам, чтобы вовремя вернуть деньги или выполнить обмен	UC-15 Обработать заявку на возврат	High
Администратор	**US-28** ЛР4 Как администратор, я хочу видеть журнал изменений правил и действий AI-ассистента, чтобы контролировать автоматические операции (ФТ-13.6)	UC-28 Просмотреть журнал действий	Medium
Платёжный шлюз	**US-29** ЛР4 Как платёжный шлюз, я передаю результат платежа вебхуком, чтобы заказ переходил в статус «оплачен» без двойного подтверждения (ФТ-15.6)	UC-06 Оплатить заказ	High
Продавец	**US-30** ЛР4 Как продавец, я хочу поручать AI-ассистенту проверку атрибутов, подготовку черновика и отправку на модерацию, чтобы готовить карточки в диалоге	UC-30 Выполнить операцию через AI-ассистента UC-10 Загрузить товар и заполнить атрибуты UC-11 Сформировать черновик описания UC-14 Отправить карточку на модерацию	Medium
Модератор	**US-31** ЛР4 Как модератор, я хочу получать от AI-ассистента сводку по карточке с найденными нарушениями, чтобы быстрее принимать решение	UC-30 Выполнить операцию через AI-ассистента UC-16 Проверить карточку в очереди модерации	Medium
Покупатель	**US-32** ЛР4 Как покупатель, я хочу оформить возврат в диалоге с ассистентом поддержки, чтобы сразу узнать условия и пункт приёма	UC-30 Выполнить операцию через AI-ассистента UC-08 Оформить возврат или обмен	Medium

1.4 Реестр вариантов использования

В реестре (таблица 4) для каждого варианта использования указаны действующие лица, обосновывающие истории, приоритет и реализация: число операций REST API и инструментов MCP-сервера. Приоритет варианта использования равен наивысшему приоритету его историй. Каждый из 30 вариантов реализован хотя бы одной операцией API или инструментом MCP; UC-17 «Заблокировать запрещённые формулировки» не имеет собственного маршрута, а включается («include») в операции правки, отправки и модерации описания — так же, как на диаграмме ЛР№3.

Таблица 4 — Реестр вариантов использования

ID	Вариант использования	Действующие лица	User Story	Приоритет	REST API	MCP	Источник
UC-01	Просмотреть ленту рекомендаций	Гость, Покупатель	US-01	High	1	—	ЛР3
UC-02	Найти товар	Гость, Покупатель	US-13, US-14	Medium	2	—	ЛР3
UC-03	Просмотреть карточку товара	Гость, Покупатель	US-02, US-20	High	2	—	ЛР3
UC-04	Подобрать размер	Гость, Покупатель	US-03, US-04	High	3	—	ЛР3
UC-05	Оформить заказ	Покупатель	US-22	High	1	—	ЛР3
UC-06	Оплатить заказ	Покупатель, Платёжный шлюз	US-22, US-29	High	3	—	ЛР3
UC-07	Отследить доставку	Покупатель, Служба доставки	US-23	Medium	2	—	ЛР3
UC-08	Оформить возврат или обмен	Покупатель, AI-ассистент	US-11, US-15, US-32	High	5	2	ЛР3
UC-09	Оставить отзыв с оценкой посадки	Покупатель	US-24	Medium	1	—	ЛР3
UC-10	Загрузить товар и заполнить атрибуты	Продавец, AI-ассистент	US-05, US-30	High	5	5	ЛР3
UC-11	Сформировать черновик описания	Система генерации описаний, Продавец	US-05, US-30	High	1	1	ЛР3
UC-12	Проверить полноту атрибутов	Система генерации описаний, Продавец	US-06	High	4	1	ЛР3
UC-13	Отредактировать описание	Продавец	US-07	Medium	4	1	ЛР3
UC-14	Отправить карточку на модерацию	Продавец	US-08, US-30	High	2	1	ЛР3
UC-15	Обработать заявку на возврат	Продавец	US-27	High	3	—	ЛР3

ID	Вариант использования	Действующие лица	User Story	Приоритет	REST API	MCP	Источник
UC-16	Проверить карточку в очереди модерации	Модератор, AI-ассистент	US-08, US-10, US-31	High	5	1	ЛР3
UC-17	Заблокировать запрещённые формулировки	Система генерации описаний	US-09	High	3	1	ЛР3
UC-18	Настроить правила генерации	Администратор	US-09, US-10	High	4	1	ЛР3
UC-19	Контролировать повторные нарушения продавца	Администратор	US-19	Medium	1	—	ЛР3
UC-20	Авторизоваться	Покупатель, Продавец, Модератор, Администратор	US-21	High	3	—	ЛР3
UC-21	Зарегистрироваться	Гость	US-21	High	1	—	ЛР4
UC-22	Удалить учётную запись	Покупатель	US-25	High	1	—	ЛР4
UC-23	Ознакомиться с условиями возврата	Гость, Покупатель	US-12	Medium	1	—	ЛР4
UC-24	Загрузить партию товаров	Продавец	US-16	Medium	1	—	ЛР4
UC-25	Управлять вариантами и остатками	Продавец	US-26	High	1	—	ЛР4
UC-26	Просмотреть аналитику описаний	Продавец	US-17	Medium	1	—	ЛР4
UC-27	Настроить обязательные атрибуты	Администратор	US-18	High	3	—	ЛР4
UC-28	Просмотреть журнал действий	Администратор	US-28	Medium	1	—	ЛР4
UC-29	Отменить заказ	Покупатель	US-23	Medium	1	—	ЛР4
UC-30	Выполнить операцию через AI-ассистента	AI-ассистент, Продавец, Модератор, Покупатель	US-30, US-31, US-32	Medium	—	13	ЛР4

Модель требований и реестр размещены на странице вики «Функциональные требования» и на одноимённой странице сайта GitHub Pages (раздел 6).

2 Физическая модель базы данных

2.1 Уточнение логической модели ЛР№3

СУБД проекта — PostgreSQL 16, как зафиксировано на диаграмме контейнеров C4 в ЛР № 3. ERD ЛР № 3 содержала 24 таблицы. При проектировании маршрутов выяснилось, что для API её недостаточно: в схеме не было учётных записей пользователей, а два внешних ключа ссылались на несуществующие таблицы. Изменения приведены в таблице 5; итоговая модель содержит 31 таблицу и 14 перечислимых типов.

Таблица 5 — Уточнения логической модели ЛР№3

Изменение	Причина
Добавлены `app_user`, `refresh_token`	в ЛР№3 покупатель и продавец не имели логина и пароля; API требует JWT-аутентификации (UC-20, UC-21) и отзыва сессии
`customer`, `seller` получили `user_id` (1:1 с `app_user`)	профиль роли отделён от учётной записи: одна схема входа для всех ролей
Добавлены `city`, `warehouse`	`customer.city_id` и `stock.warehouse_id` в ЛР№3 ссылались на отсутствующие таблицы
Добавлена `comment_template`	шаблоны комментариев редактирует администратор (AC-10.5), значит, это данные, а не константы кода
Добавлена `seller_violation`	журнал блокировок для контроля повторных нарушений (AC-09.6) упоминался в ЛР№3, но не был таблицей
Добавлена `audit_log`	журнал изменений правил (ФТ-13.6) и действий AI-агента через MCP
`payment.status`: строка → `payment_status_enum`	ФТ-11.2 требует ENUM для перечислений
`payment.external_id` — уникальный	идемпотентная обработка повторного вебхука (ФТ-15.6)
`moderation_task.closed_at` и частичный уникальный индекс	не более одной открытой задачи модерации на описание
`description.current_version`, `description_version.prompt_version`	откат к версии (AC-05.7) и сравнение качества по версии промпта (US-17)
`CHECK`-ограничения	summary ≤ 90 символов (AC-01.2), комментарий обязателен при возврате карточки (AC-10.3), статус `blocked` только с причиной (US-09), резерв не больше остатка (ФТ-11.5), формат телефона `+375XXXXXXXXX` и УНП

2.2 Логическая схема

Логическая схема построена в форме «Сущность — Таблица БД — Поле — Смысл поля — Тип данных». Типы указаны независимо от СУБД (UUID, VARCHAR(n), DECIMAL(p,s), ENUM(...)), ключи и необязательные поля отмечены в столбце «Смысл поля». Полная схема (172 поля) приведена в приложении А; в таблице 6 — фрагмент для ядра продукта, сущностей «Описание товара» и «Версия описания».

Таблица 6 — Фрагмент логической схемы (домен «Описания и модерация»)

Сущность	Таблица БД	Поле	Смысл поля	Тип данных
Описание товара	`description`	`id`	уникальный идентификатор описания PK	UUID
		`product_id`	товар (не более одного описания) FK → product, UK	UUID
		`status`	статус модерации	ENUM(incomplete, draft, pending_moderation, blocked, returned, published)
		`confidence`	уровень уверенности генератора NULL	ENUM(low, medium, high)
		`block_reason`	найденная запрещённая формулировка (US-09) NULL	TEXT
		`current_version`	номер актуальной версии	INTEGER
		`updated_at`	дата последнего изменения	TIMESTAMP
Версия описания	`description_version`	`id`	уникальный идентификатор версии PK	UUID
		`description_id`	описание FK → description	UUID
		`number`	порядковый номер версии	INTEGER
		`kind`	тип: полная версия или summary (AC-05.5)	ENUM(full, summary)
		`body`	текст версии	TEXT
		`author`	автор: generator или идентификатор продавца	VARCHAR(64)
		`prompt_version`	версия шаблона промпта (US-17) NULL	VARCHAR(16)
		`created_at`	дата создания	TIMESTAMP

Смысл каждого поля записан в самом SQL-файле командами `COMMENT ON COLUMN`. Скрипт `tools/gen_db_docs.py` разбирает `schema.sql` парсером PostgreSQL (библиотека `pglast`) и по этим комментариям строит логическую схему для вики и приложения А, поэтому схема не может разойтись с SQL.

2.3 Физическая модель

Физическая модель представлена ER-диаграммами в нотации «вороньих лапок» (рисунки 1–4). Диаграммы разделены по доменам — ограниченным контекстам Event Storming из ЛР№3; таблицы соседних доменов показаны только первичным ключом. Диаграммы также генерируются из `schema.sql` (Mermaid), обязательность связи определяется по `NOT NULL` внешнего ключа, связь «один к одному» — по уникальности.



Рисунок 1 — ER-диаграмма домена «Пользователи и доступ» и журнала действий

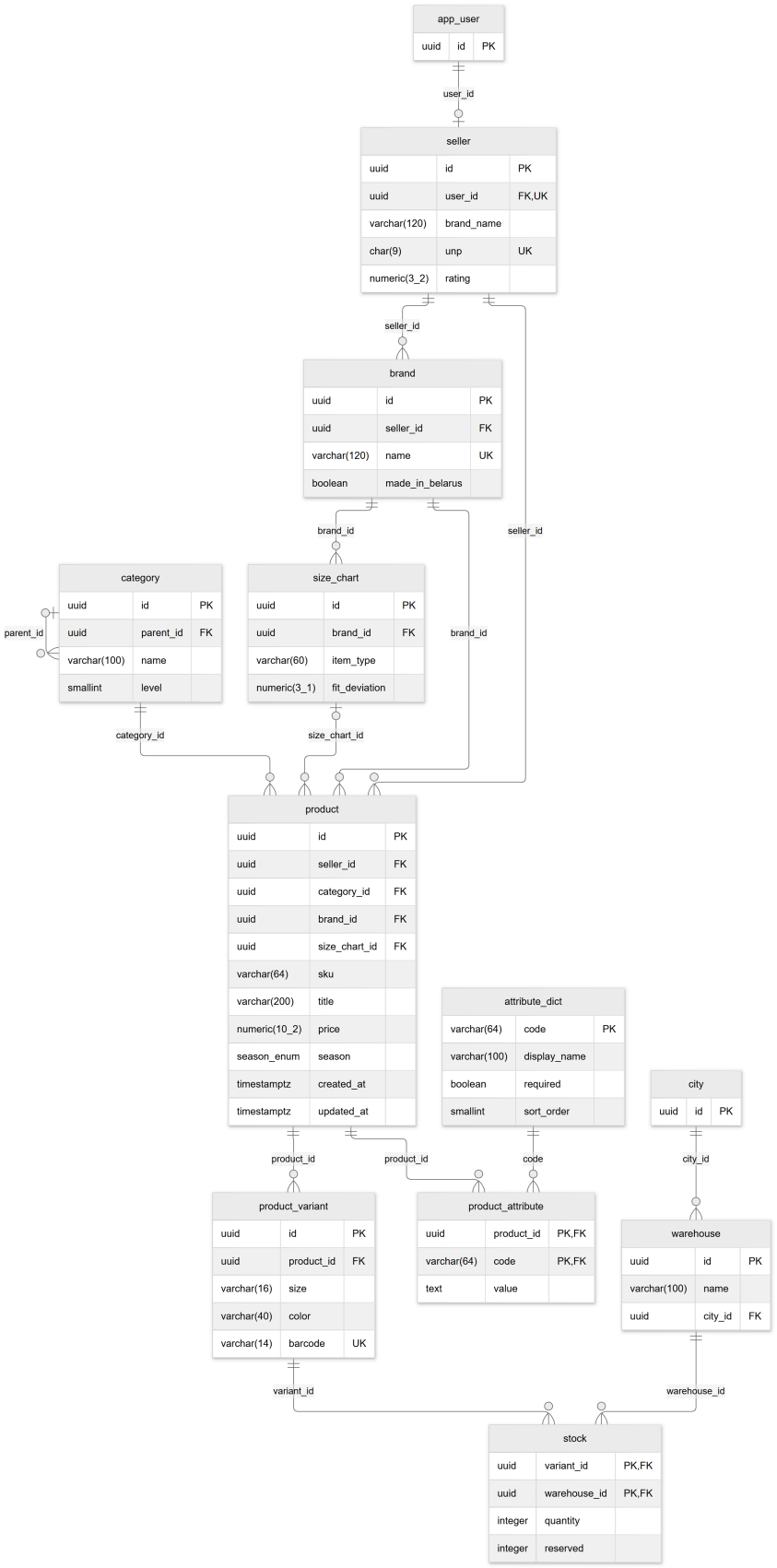


Рисунок 2 — ER-диаграмма домена «Каталог»

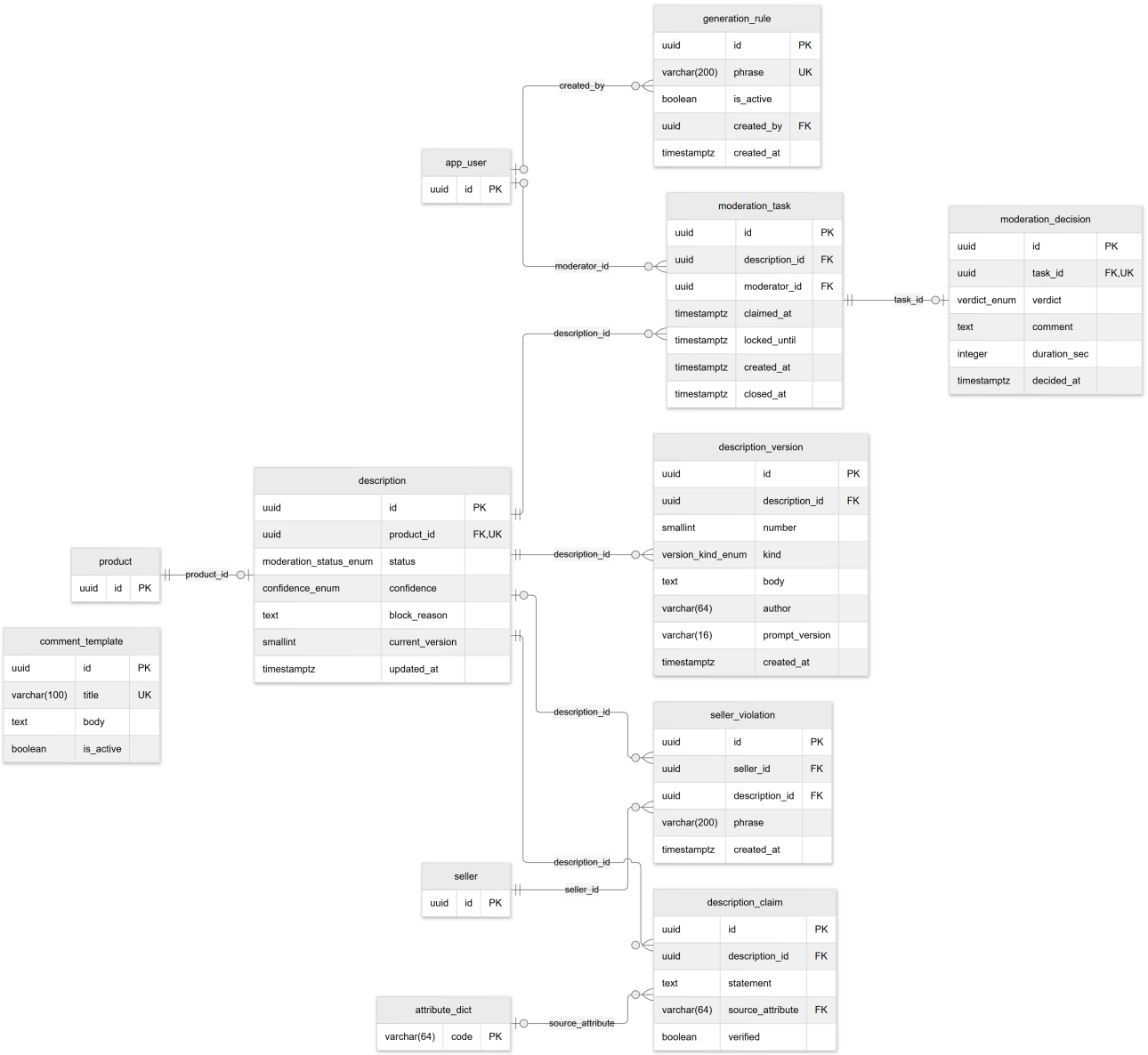


Рисунок 3 — ER-диаграмма домена «Описания и модерация»



Рисунок 4 — ER-диаграмма домена «Заказы, возвраты, отзывы»

Решения физического уровня:

1. **Первичные ключи — `uuid` с `gen_random_uuid()`** (встроена в PostgreSQL начиная с версии 13, расширения не нужны). Идентификаторы попадают в URL API (`/products/{productId}`) и не должны позволять перебором оценить объём каталога. Исключение — `audit_log.id` типа `bigint IDENTITY`: журнал только дописывается, и порядок записей важнее непредсказуемости.
2. **ENUM-типы совпадают с перечислениями OpenAPI** — сервер не преобразует значения между слоями.
3. **Каскадное удаление только внутри агрегата**: удаление товара удаляет варианты, атрибуты, описание и его версии; удалить вариант, который есть в заказе, нельзя — `order_item` ссылается на него без каскада.
4. **Индексы под маршруты**: `description(status)` — список товаров продавца по статусу, частичный индекс `moderation_task WHERE closed_at IS NULL` — очередь модерации, `orders(customer_id, created_at DESC)` — «Мои заказы».
5. **Полнотекстового индекса нет**: по решению ЛР № 3 поиск вынесен в OpenSearch.

Фрагмент `schema.sql` с таблицами ядра продукта:

```
CREATE TABLE description (
    id              uuid                    PRIMARY KEY DEFAULT gen_random_uuid(),
    product_id      uuid                    NOT NULL UNIQUE REFERENCES product (id) ON DELETE
CASCADE,
    status          moderation_status_enum NOT NULL DEFAULT 'draft',
    confidence      confidence_enum,
    block_reason    text,
    current_version smallint                NOT NULL DEFAULT 1,
    updated_at      timestamptz             NOT NULL DEFAULT now(),
    CONSTRAINT ck_description_blocked CHECK (status <> 'blocked' OR block_reason IS NOT NULL)
);
COMMENT ON TABLE  description                 IS 'Описание товара|Описание с жизненным циклом
модерации (ФТ-11.3)';
COMMENT ON COLUMN description.id              IS 'уникальный идентификатор описания';
COMMENT ON COLUMN description.product_id      IS 'товар (не более одного описания)';
COMMENT ON COLUMN description.status          IS 'статус модерации';
COMMENT ON COLUMN description.confidence      IS 'уровень уверенности генератора';
COMMENT ON COLUMN description.block_reason    IS 'найденная запрещённая формулировка (US-09)';
COMMENT ON COLUMN description.current_version IS 'номер актуальной версии';
COMMENT ON COLUMN description.updated_at      IS 'дата последнего изменения';

CREATE TABLE description_version (
    id              uuid              PRIMARY KEY DEFAULT gen_random_uuid(),
    description_id  uuid              NOT NULL REFERENCES description (id) ON DELETE CASCADE,
    number          smallint          NOT NULL CHECK (number > 0),
    kind            version_kind_enum NOT NULL,
    body            text              NOT NULL,
    author          varchar(64)       NOT NULL,
    prompt_version  varchar(16),
    created_at      timestamptz       NOT NULL DEFAULT now(),
    CONSTRAINT uq_version UNIQUE (description_id, number, kind),
    CONSTRAINT ck_summary_len CHECK (kind <> 'summary' OR length(body) <= 90)
);
```

2.4 Проверка схемы

Схема проверена в настоящей СУБД. Скрипт `db/verify_schema.py` поднимает временный экземпляр PostgreSQL 16.2 (пакет `pgserver`), применяет `schema.sql` и `seed.sql`, а затем выполняет девять заведомо некорректных операций и проверяет, что каждая отклонена ограничением схемы.

```
[OK] schema.sql применён без ошибок
[OK] seed.sql применён без ошибок
     таблиц: 31, ENUM-типов: 14
     PostgreSQL 16.2 on x86_64-pc-mingw64
[OK] отклонено: второе описание у товара (ФТ-11.3)
[OK] отклонено: статус blocked без причины блокировки (US-09)
[OK] отклонено: возврат карточки без комментария (АС-10.3)
[OK] отклонено: summary длиннее 90 символов (АС-01.2)
[OK] отклонено: резерв больше остатка (ФТ-11.5)
[OK] отклонено: значение вне перечисления (ФТ-11.2)
[OK] отклонено: телефон не в формате +375
[OK] отклонено: вторая открытая задача модерации на одно описание
[OK] отклонено: повторный отзыв того же покупателя
```

Проверка нашла ошибку в первой версии схемы: ограничение `CHECK (verdict = 'approve' OR length(trim(comment)) > 0)` пропускало возврат карточки с `comment = NULL`, потому что выражение с `NULL` даёт `NULL`, а `CHECK` с результатом `NULL` строку не отклоняет. Ограничение дополнено условием `comment IS NOT NULL`.

3 Маршруты и конечные точки API

3.1 Принципы проектирования

Маршруты спроектированы от сущностей физической модели по правилам REST, рассмотренным в материалах GetAnalyst (таблица 7).

Таблица 7 — Соглашения API

Вопрос	Решение	Обоснование
Версия	в пути: `/v1`	несовместимые изменения — только в новой версии
Именование	существительные во множественном числе, kebab-case	`/seller/products`, `/return-policies`
Действия вне CRUD	подресурс-глагол: `/description/submit`, `/tasks/{id}/claim`, `/orders/{id}/cancel`	смена состояния с проверками не сводится к `PUT` всего ресурса
Разделение ролей	префиксы `/seller`, `/moderation`, `/admin`	по ЛР№3 кабинеты продавца и модератора — маршруты одного веб-приложения; префикс упрощает авторизацию на шлюзе
Чужие ресурсы	404, а не 403	не раскрывать существование чужого товара или заказа
Ошибки	`application/problem+json` (RFC 9457)	единый формат для всех клиентов; поле `missing` перечисляет недостающие атрибуты (АС-06.2)
Покупательский контур	в схемах карточки нет `confidence` и `claims`	ограничение ОГР-2: термины генерации покупателю не показываются

3.2 Маршруты

В таблице 8 каждой сущности сопоставлены таблицы БД, которые читает или изменяет маршрут, и сами маршруты. Скрипт проверки подтверждает, что все 45 маршрутов спецификации присутствуют в таблице, а все упомянутые таблицы существуют в `schema.sql`.

Таблица 8 — Сущности, таблицы БД и маршруты

Сущность	Таблица БД	Маршрут

Учётная запись, токен	`app_user`, `refresh_token`	`/auth/register`
		`/auth/login`
		`/auth/refresh`
		`/auth/logout`

Покупатель	`customer`, `size_params`	`/customers/me` `/customers/me/size-params`
Лента рекомендаций	`product`, `description_version`	`/feed`
Категория	`category`	`/categories`
Товар (витрина)	`product`, `product_variant`, `stock`, `description_version`	`/products` `/products/{productId}`

Сущность	Таблица БД	Маршрут
Рекомендация размера	size_params, size_chart, review	/products/{productId}/size-recommendation
Правила возврата	return_policy	/return-policies
Отзыв	review	/products/{productId}/reviews
Товар продавца	product, product_attribute, attribute_dict	/seller/products /seller/products/batch /seller/products/{productId} /seller/products/{productId}/attribute-check
Вариант товара, остаток	product_variant, stock, warehouse	/seller/products/{productId}/variants
Описание товара	description, description_version, description_claim, moderation_task	/seller/products/{productId}/description /seller/products/{productId}/description/generate /seller/products/{productId}/description/rollback /seller/products/{productId}/description/submit
Подсказки о посадке	review	/seller/products/{productId}/fit-hints
Заявка на возврат (продавец)	return_request	/seller/returns /seller/returns/{returnId}/decision
Аналитика описаний	orders, order_item, return_request	/seller/analytics
Задача модерации	moderation_task, moderation_decision, description	/moderation/tasks /moderation/tasks/{taskId}/claim /moderation/tasks/{taskId}/decision /moderation/tasks/batch-approve
Шаблон комментария	comment_template	/moderation/comment-templates
Правило генерации	generation_rule	/admin/generation-rules /admin/generation-rules/{ruleId}
Справочник атрибутов	attribute_dict	/admin/attributes /admin/attributes/{code}
Нарушение продавца	seller_violation, seller	/admin/violations
Журнал действий	audit_log	/admin/audit-log
Заказ	orders, order_item, stock	/orders /orders/{orderId} /orders/{orderId}/cancel
Платёж	payment	/orders/{orderId}/payment /payments/webhook
Заявка на возврат	return_request, return_photo	/returns /returns/{returnId} /returns/{returnId}/photos

3.3 Варианты использования и конечные точки

Таблица 9 сопоставляет варианты использования с конечными точками — HTTP-методами маршрутов. Она построена из спецификации: в описании каждой операции OpenAPI указан реализуемый UC, действующее лицо определяется тегом операции, требование аутентификации — блоком `security`. «Необязательна» означает, что операция доступна гостю, но с токеном возвращает персонализированный результат (лента, рекомендация размера).

Таблица 9 — Варианты использования и конечные точки

Актор	Use Case	Маршрут	HTTP-запрос	Аутентификация
Гость, Покупатель	UC-01 Просмотреть ленту рекомендаций	`/feed`	GET	необязательна
Гость, Покупатель	UC-02 Найти товар	`/categories`	GET	нет
Гость, Покупатель	UC-02 Найти товар	`/products`	GET	нет
Гость, Покупатель	UC-03 Просмотреть карточку товара	`/products/{productId}`	GET	нет
Гость, Покупатель	UC-03 Просмотреть карточку товара	`/products/{productId}/reviews`	GET	нет
Покупатель	UC-04 Подобрать размер	`/customers/me`	GET	да (buyer)
Покупатель	UC-04 Подобрать размер	`/customers/me/size-params`	PUT	да (buyer)
Гость, Покупатель	UC-04 Подобрать размер	`/products/{productId}/size-recommendation`	GET	необязательна
Покупатель	UC-05 Оформить заказ (+ UC-06)	`/orders`	POST	да (buyer)
Покупатель	UC-06 Оплатить заказ	`/orders/{orderId}/payment`	POST	да (buyer)
Платёжный шлюз	UC-06 Оплатить заказ	`/payments/webhook`	POST	нет
Покупатель	UC-07 Отследить доставку (+ UC-08)	`/orders`	GET	да (buyer)
Покупатель	UC-07 Отследить доставку	`/orders/{orderId}`	GET	да (buyer)
Покупатель	UC-08 Оформить возврат или обмен	`/returns`	GET	да (buyer)
Покупатель	UC-08 Оформить возврат или обмен (+ UC-15)	`/returns`	POST	да (buyer)
Покупатель	UC-08 Оформить возврат или обмен	`/returns/{returnId}`	GET	да (buyer)
Покупатель	UC-08 Оформить возврат или обмен	`/returns/{returnId}/photos`	POST	да (buyer)
Покупатель	UC-09 Оставить отзыв с оценкой посадки	`/products/{productId}/reviews`	POST	да (buyer)

Актор	Use Case	Маршрут	HTTP-запрос	Аутентификация
Продавец	UC-10 Загрузить товар и заполнить атрибуты (+ UC-14)	`/seller/products`	GET	да (seller)
Продавец	UC-10 Загрузить товар и заполнить атрибуты (+ UC-12)	`/seller/products`	POST	да (seller)
Продавец	UC-10 Загрузить товар и заполнить атрибуты	`/seller/products/{productId}`	DELETE	да (seller)
Продавец	UC-10 Загрузить товар и заполнить атрибуты	`/seller/products/{productId}`	GET	да (seller)
Продавец	UC-10 Загрузить товар и заполнить атрибуты (+ UC-12)	`/seller/products/{productId}`	PUT	да (seller)
Продавец	UC-11 Сформировать черновик описания (+ UC-12)	`/seller/products/{productId}/description/generate`	POST	да (seller)
Продавец	UC-12 Проверить полноту атрибутов (+ UC-27)	`/seller/products/{productId}/attribute-check`	GET	да (seller)
Продавец	UC-13 Отредактировать описание	`/seller/products/{productId}/description`	GET	да (seller)
Продавец	UC-13 Отредактировать описание (+ UC-17)	`/seller/products/{productId}/description`	PUT	да (seller)
Продавец	UC-13 Отредактировать описание	`/seller/products/{productId}/description/rollback`	POST	да (seller)
Продавец	UC-13 Отредактировать описание	`/seller/products/{productId}/fit-hints`	GET	да (seller)
Продавец	UC-14 Отправить карточку на модерацию (+ UC-17)	`/seller/products/{productId}/description/submit`	POST	да (seller)
Продавец	UC-15 Обработать заявку на возврат	`/seller/returns`	GET	да (seller)
Продавец	UC-15 Обработать заявку на возврат	`/seller/returns/{returnId}/decision`	POST	да (seller)
Модератор	UC-16 Проверить карточку в очереди модерации	`/moderation/comment-templates`	GET	да (moderator)
Модератор	UC-16 Проверить карточку в очереди модерации	`/moderation/tasks`	GET	да (moderator)
Модератор	UC-16 Проверить карточку в очереди модерации	`/moderation/tasks/batch-approve`	POST	да (moderator)
Модератор	UC-16 Проверить карточку в очереди модерации	`/moderation/tasks/{taskId}/claim`	POST	да (moderator)
Модератор	UC-16 Проверить карточку в очереди модерации (+ UC-17)	`/moderation/tasks/{taskId}/decision`	POST	да (moderator)
Администратор	UC-18 Настроить правила генерации	`/admin/generation-rules`	GET	да (admin)
Администратор	UC-18 Настроить правила генерации	`/admin/generation-rules`	POST	да (admin)

Актор	Use Case	Маршрут	HTTP-запрос	Аутентификация
Администратор	UC-18 Настроить правила генерации	`/admin/generation-rules/{ruleId}`	DELETE	да (admin)
Администратор	UC-18 Настроить правила генерации	`/moderation/comment-templates`	POST	да (admin)
Администратор	UC-19 Контролировать повторные нарушения продавца	`/admin/violations`	GET	да (admin)
Пользователь (все роли)	UC-20 Авторизоваться	`/auth/login`	POST	нет
Пользователь (все роли)	UC-20 Авторизоваться	`/auth/logout`	POST	да
Пользователь (все роли)	UC-20 Авторизоваться	`/auth/refresh`	POST	нет
Пользователь (все роли)	UC-21 Зарегистрироваться	`/auth/register`	POST	нет
Покупатель	UC-22 Удалить учётную запись	`/customers/me`	DELETE	да (buyer)
Гость, Покупатель	UC-23 Ознакомиться с условиями возврата	`/return-policies`	GET	нет
Продавец	UC-24 Загрузить партию товаров	`/seller/products/batch`	POST	да (seller)
Продавец	UC-25 Управлять вариантами и остатками	`/seller/products/{productId}/variants`	PUT	да (seller)
Продавец	UC-26 Просмотреть аналитику описаний	`/seller/analytics`	GET	да (seller)
Администратор	UC-27 Настроить обязательные атрибуты	`/admin/attributes`	GET	да (admin)
Администратор	UC-27 Настроить обязательные атрибуты	`/admin/attributes/{code}`	PUT	да (admin)
Администратор	UC-28 Просмотреть журнал действий	`/admin/audit-log`	GET	да (admin)
Покупатель	UC-29 Отменить заказ	`/orders/{orderId}/cancel`	POST	да (buyer)

4 Проектирование и тестирование API

4.1 Спецификация OpenAPI

Спецификация `api/openapi.yaml` написана на OpenAPI 3.0.3 по образцу Swagger Petstore: 45 маршрутов, 55 операций, 12 тегов по ролям, 53 схемы данных, 6 переиспользуемых ответов-ошибок. В описании каждой операции указаны реализуемый вариант использования, истории и критерии приёмки ЛР№3 — по ним строились таблицы раздела 3 и тесты. Фрагмент спецификации — операция отправки описания на модерацию:

```
  /seller/products/{productId}/description/submit:
    parameters: [{ $ref: '#/components/parameters/ProductId' }]
    post:
      tags: [seller-descriptions]
      operationId: submitDescription
      summary: Отправить на модерацию
      description: UC-14 «include» UC-17 · US-08, US-09 (AC-09.1). Создаёт задачу в очереди
модерации.
      responses:
        '200':
          description: Описание поставлено в очередь модерации
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Description' }
        '401': { $ref: '#/components/responses/Unauthorized' }
        '404': { $ref: '#/components/responses/NotFound' }
        '409':
          description: Описание заблокировано до правки или уже на модерации
          content:
            application/problem+json:
              schema: { $ref: '#/components/schemas/Problem' }
              example:
                type: 'https://api.marketshop.by/problems/blocked'
                title: Описание заблокировано до правки
                status: 409
                detail: 'Найдена запрещённая формулировка: «лучший»'
```

Спецификация проверена двумя независимыми валидаторами: `openapi-spec-validator` — «OK», Redocly CLI — без ошибок и предупреждений. Интерактивная документация сформирована Swagger UI (рисунок 5) и размещена на странице `docs/api/index.html` сайта GitHub Pages.

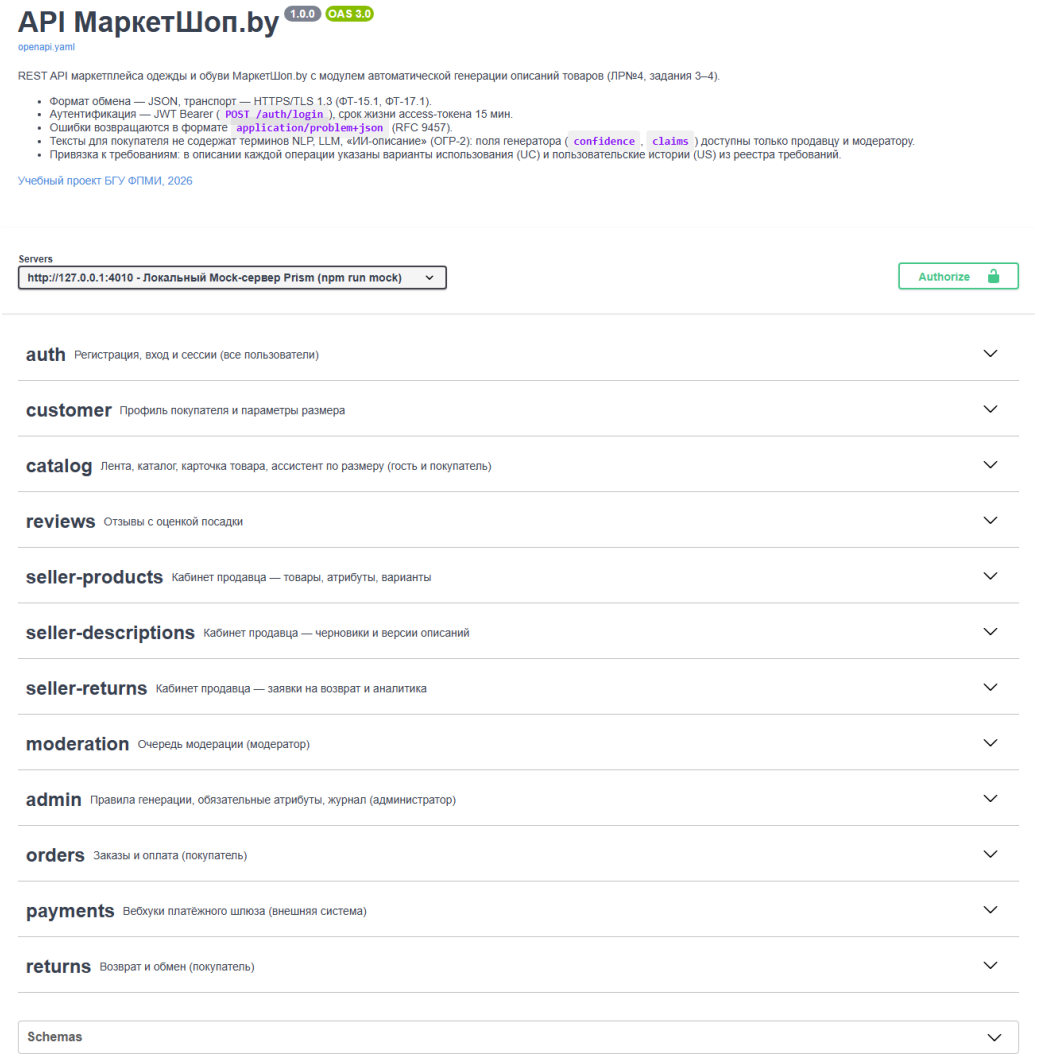


Рисунок 5 — Документация API МаркетШоп.by в Swagger UI

4.2 Тела POST/PUT-запросов и HTTP-статусы

Для каждого POST- и PUT-запроса определено тело (таблица 10) и перечень ответов с описанием смысла каждого кода. Схемы тел содержат ограничения, которые mock-сервер проверяет при каждом запросе: обязательные поля, `minLength`, `pattern` (телефон, УНП, штрихкод), `enum`, `minItems`/`maxItems` (партия — не более 20 позиций, АС-05.2).

Таблица 10 — Тела POST/PUT-запросов и коды ответов

Операция	Тело запроса	Обязательные поля	Коды ответов
POST /auth/register	RegisterRequest	login, password, role	201, 400, 409, 422
POST /auth/login	LoginRequest	login, password	200, 400, 401
POST /auth/refresh	объект	refresh_token	200, 401
POST /auth/logout	— (действие над ресурсом из пути)	—	204, 401
PUT /customers/me/size-params	SizeParams	—	200, 400, 401, 422
POST /products/{productId}/reviews	ReviewCreate	rating	201, 400, 401, 403, 409
POST /seller/products	SellerProductInput	sku, title, category_id, brand_id, price, attributes	201, 400, 401, 403, 409, 422
POST /seller/products/batch	объект	items	200, 400, 401, 413
PUT /seller/products/{productId}	SellerProductInput	sku, title, category_id, brand_id, price, attributes	200, 400, 401, 404, 422
PUT /seller/products/{productId}/variants	массив VariantInput	size, color, stock	200, 400, 401, 404, 409
PUT /seller/products/{productId}/description	DescriptionEdit	full	200, 400, 401, 404, 422
POST /seller/products/{productId}/description/generate	объект (необязательно)	—	201, 401, 404, 422, 503
POST /seller/products/{productId}/description/rollback	объект	version	200, 401, 404
POST /seller/products/{productId}/description/submit	— (действие над ресурсом из пути)	—	200, 401, 404, 409
POST /seller/returns/{returnId}/decision	ReturnDecision	decision	200, 401, 404, 409, 422
POST /moderation/tasks/{taskId}/claim	— (действие над ресурсом из пути)	—	200, 401, 404, 409
POST /moderation/tasks/{taskId}/decision	ModerationDecisionInput	verdict	200, 401, 404, 409, 422
POST /moderation/tasks/batch-approve	объект	task_ids	200, 401, 403, 422

Операция	Тело запроса	Обязательные поля	Коды ответов
POST /moderation/comment-templates	CommentTemplateInput	title, body	201, 401, 403, 409
POST /admin/generation-rules	объект	phrase	201, 400, 401, 403, 409
PUT /admin/attributes/{code}	AttributeDefinition	code, display_name, required	200, 400, 401, 403
POST /orders	OrderInput	items, payment_method, delivery_method, delivery_address	201, 400, 401, 409, 422
POST /orders/{orderId}/cancel	— (действие над ресурсом из пути)	—	200, 401, 404, 409
POST /orders/{orderId}/payment	— (действие над ресурсом из пути)	—	201, 401, 404, 409
POST /payments/webhook	PaymentWebhook	external_id, order_number, status, amount	200, 400, 401
POST /returns	ReturnInput	order_item_id, reason	201, 400, 401, 404, 422
POST /returns/{returnId}/photos	multipart/form-data	file	201, 401, 404, 413

Смысл кодов ответов един для всего API (таблица 11). Отдельно разведены 400 и 422: 400 — запрос не разобран (неверный JSON, тип параметра, нет обязательного заголовка), 422 — запрос корректен, но нарушает правило предметной области (неполные атрибуты, истёк срок возврата). Клиенту это сообщает, что исправлять нужно данные, а не код запроса.

Таблица 11 — Смысл HTTP-статусов в API

Код	Смысл в API МаркетШоп.by	Операций
200	Успех, тело содержит результат	41
201	Ресурс создан	10
204	Успех без тела	4
400	Синтаксическая ошибка запроса или неверный параметр	16
401	Нет или недействителен JWT / подпись	47
403	Роль не позволяет выполнить операцию	14
404	Ресурс не найден или чужой	24
409	Конфликт состояния (дубликат, блокировка, статус не позволяет)	15
413	Превышен размер (партия > 20, файл > 10 МБ)	2
422	Нарушено бизнес-правило или схема тела	12
429	Превышен лимит запросов	3
503	Внешний сервис генерации недоступен	1

4.3 Mock-сервер и коллекция запросов

Для работы с API выбран **Bruno** — клиент с поддержкой OpenAPI, который хранит коллекцию запросов в виде текстовых файлов прямо в репозитории и не требует облачной учётной записи. Коллекция `api/bruno` открывается в приложении Bruno и запускается из командной строки (Bruno CLI 4.3).

Mock-сервер — **Stoplight Prism 5.16**, запускаемый локально командой `npm run mock` из того же `openapi.yaml`. Выбор объясняется тем, что Prism не только возвращает примеры ответов из спецификации, но и проверяет входящие запросы по ней: запрос без токена получает 401, тело с нарушением схемы — 422, неверный параметр — 400. Mock-сервер и документация не могут разойтись, потому что строятся из одного файла. Заголовок `Prefer: code=409` возвращает любой описанный в спецификации ответ, что позволяет проверить ветви ошибок. Та же спецификация пригодна для импорта в SwaggerHub с автоматическим mock-сервером (API Auto Mocking).

Таблица 12 — Структура коллекции Bruno

Папка	Запросов	Негативных	Проверяемые требования
01 Авторизация	4	2	UC-20, UC-21; 400 при неполном теле, 401 при неверном пароле
02 Каталог	6	1	AC-01.2 (summary ≤ 90), AC-02.2 (подблоки описания), AC-02.6 (нет терминов генерации), US-13, AC-12.1
03 Покупатель	3	1	US-04, US-24; 401 без токена
04 Продавец	10	3	AC-06.1, AC-06.2 (список недостающих полей), US-05, US-07, AC-09.1
05 Модерация	6	2	AC-08.1, AC-09.2, AC-08.7 (захват задачи), enum `verdict`
06 Администратор	3	0	AC-09.4, US-18, ФТ-13.6
07 Заказы	7	3	US-22, US-23, ФТ-15.6; `minItems`, обязательная подпись вебхука
08 Возвраты	4	1	AC-11.5 (пункт и срок сдачи), US-15 (понятная причина отказа), UC-15
Итого	**43**	**13**	

Каждый запрос проверяет код ответа (блок `assert`) и в ключевых случаях содержимое тела (блок `tests`). Пример — запрос загрузки товара с неполными атрибутами:

```
assert {
  res.status: eq 201
}

tests {
  test("статус 201", function () {
    expect(res.getStatus()).to.equal(201);
  });
  test("тело ответа соответствует требованиям", function () {
    expect(res.body.description_status).to.equal("incomplete");
    expect(res.body.attribute_check.missing.map(m => m.display_name)).to.include("Размерная
сетка");
```

```
  });
}
```

4.4 Результаты тестирования

Коллекция запущена против mock-сервера командой `npm run test:api`. Результат: 43 запроса, 65 тестов и 43 проверки — все пройдены, ошибок нет (рисунок 6). Логи — `api/results/bruno-run.log`, HTML-отчёт — `api/results/bruno-report.html`.

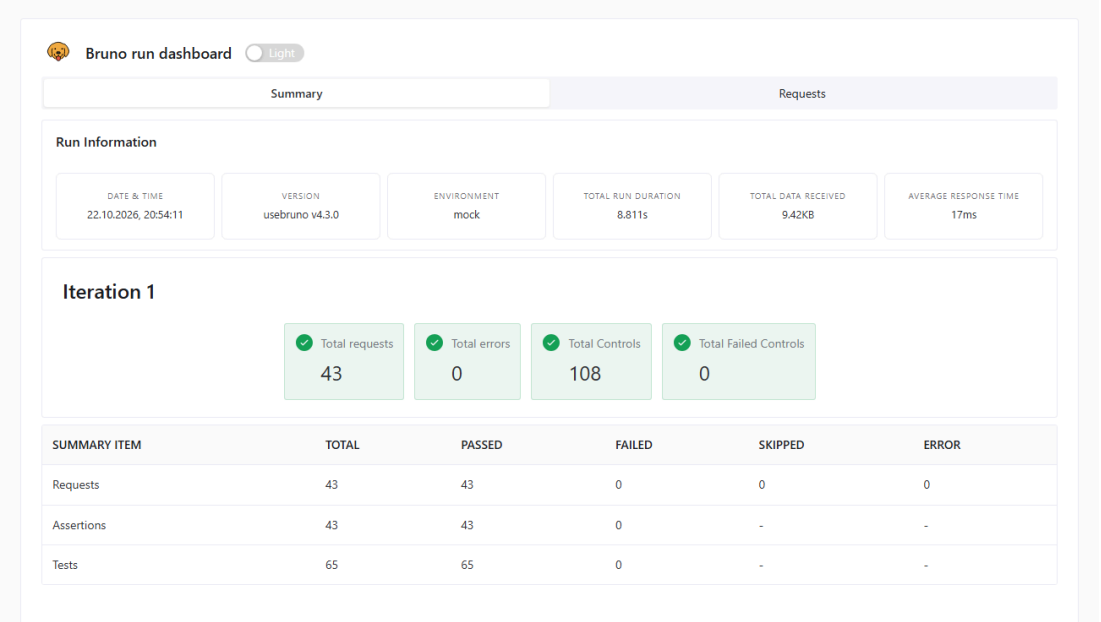


Рисунок 6 — Отчёт Bruno CLI о прогоне коллекции против mock-сервера

Примеры ответов mock-сервера (полностью — на странице вики «API проекта»):

```
POST /seller/products/90000000-0000-4000-8000-000000000001/description/submit
Authorization: Bearer <token>

HTTP/1.1 409 Conflict
Content-Type: application/problem+json
{"type": "https://api.marketshop.by/problems/blocked", "title": "Описание заблокировано до правки",
 "status": 409, "detail": "Найдена запрещённая формулировка: «лучший»"}

PUT /customers/me/size-params
Content-Type: application/json
{"height_cm": 168}

HTTP/1.1 401 Unauthorized
Content-Type: application/problem+json
{"type": "https://api.marketshop.by/problems/unauthorized", "title": "Требуется вход в систему",
"status": 401}
```

При подготовке тестов найдена и исправлена несогласованность в самой спецификации: в примере запроса загрузки товара передавался сезон, а пример ответа называл «Сезон» среди недостающих атрибутов. Пример ответа исправлен, тест теперь проверяет, что недостающей названа только «Размерная сетка».

5 MCP-сервер

5.1 Назначение и анализ примеров

Ключевая функция продукта — генерация описаний: описание создаёт внутренний сервис, но готовят карточку и проверяют черновик люди — продавец и модератор. MCP-сервер `marketshop-assistant` даёт им AI-ассистента, который выполняет эти операции по просьбе на естественном языке, а сервер не позволяет ассистенту нарушить бизнес-правила. Третий сценарий — возвраты: сложность возврата — барьер №1 по данным опроса ЛР№1 (39,1 %). Соответствие примерам из методических указаний показано в таблице 13.

Таблица 13 — Соответствие примеров MCP-серверов и решений проекта

Пример	Решение в проекте
Интернет-магазин: Tools `create_order`, `cancel_order`, `check_inventory`	`create_product`, `delete_product`, `check_attributes`, `create_return_request`
Resources `catalog://electronics`, `orders://user/{id}/history`	`catalog://products`, `orders://{order_number}/summary`
Prompt `refund_calculation`: сначала прочитать правила возврата	Prompt `return_request_assistant`: сначала сводка заказа и `returns://policy/{category}`, затем инструменты
Валидация: нельзя отменить доставленный заказ	нельзя вернуть неполученный заказ, после истечения срока, из невозвратной категории; нельзя одобрить описание с запрещённой формулировкой; нельзя отправить заблокированное описание
Tool требует строгий `tracking_number` из ресурса	номер заказа проверяется шаблоном `^MS-\d{4}-\d{6}$`, промпты требуют брать идентификаторы только из ресурсов
Клиника: логирование действий агента с конфиденциальными данными	журнал `audit_log` с результатом `ok`/`rejected`; телефон в ресурсах маскируется, в журнал персональные данные не пишутся

5.2 Архитектура

Архитектура показана на рисунке 7. Сервер реализован на официальном MCP Python SDK 2.3 (класс `MCPServer`), транспорт — stdio. Слой `server.py` объявляет примитивы MCP, слой `service.py` содержит бизнес-правила и не зависит от MCP — те же правила описаны в REST API (US-06, US-09, US-11, US-15), поэтому ассистент и веб-интерфейс ведут себя одинаково. Данные хранятся в mock-базе SQLite из 17 таблиц — подмножестве физической модели раздела 2, где ENUM заменены `CHECK`-ограничениями.

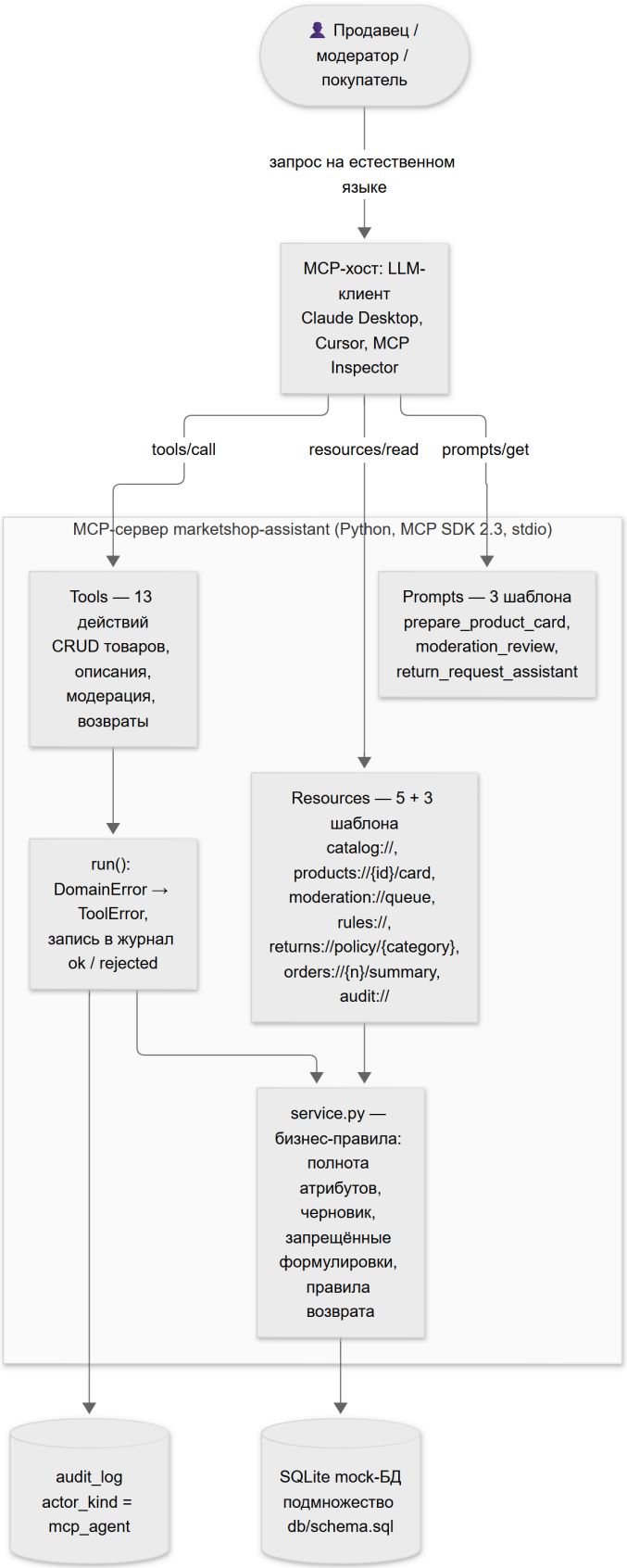


Рисунок 7 — Архитектура MCP-сервера

Сущности сервера: товар, атрибут и справочник атрибутов, описание, версия описания, утверждение, правило генерации, задача и решение модерации, покупатель, заказ, позиция заказа, правила возврата, заявка на возврат, журнал действий.

5.3 Tools, Resources и Prompts

Таблица 14 — Инструменты MCP-сервера

Инструмент	CRUD	Аннотация	Бизнес-валидация
`list_products(status?)`	Read	read-only	`status` — перечисление статусов
`get_product(product_id)`	Read	read-only	UUID по шаблону
`create_product(...)`	Create	write	цена > 0, известные коды атрибутов, уникальный артикул
`update_product_attributes(...)`	Update	write	нельзя менять товар, описание которого на модерации
`delete_product(product_id, confirm)`	Delete	destructive	нужен `confirm=true`; товар из заказов не удаляется
`check_attributes(product_id)`	Read	read-only	—
`generate_description(product_id)`	Create	write	отказ при неполных обязательных атрибутах (AC-06.1)
`edit_description(...)`	Update	write	новая версия; запрещённая формулировка → `blocked`
`submit_for_moderation(product_id)`	Update	write	заблокированное или неполное описание не отправляется
`moderate_description(...)`	Update	write	возврат без комментария невозможен; текст с запрещённой формулировкой не одобряется
`add_forbidden_phrase(phrase)`	Create	write	дубликаты отклоняются; правило действует сразу
`check_return_eligibility(...)`	Read	read-only	понятная причина отказа (US-15)
`create_return_request(...)`	Create	write	заказ вручён, срок не истёк, категория возвратная, заявка не дублируется

JSON-схемы входных параметров SDK строит из аннотаций типов Python (`Annotated`, `Literal`, `pydantic.Field`), поэтому описание и проверка аргументов не расходятся. Схема инструмента `create_return_request`, полученная от сервера запросом `tools/list`:

```
{"type": "object",
 "properties": {
   "order_number": {"type": "string", "pattern": "^MS-\\d{4}-\\d{6}$", "description": "Номер заказа
вида MS-2026-000154"},
   "reason": {"type": "string", "enum": ["size_mismatch", "not_as_described", "defect",
"changed_mind"]},
   "is_exchange": {"type": "boolean", "default": false},
   "order_item_id": {"anyOf": [{"type": "string"}, {"type": "null"}], "default": null}},
 "required": ["order_number", "reason"]}
```

Таблица 15 — Ресурсы MCP-сервера

URI	Вид	Данные
`catalog://products`	статический	товары продавца со статусами описаний
`products://{product_id}/card`	шаблон	атрибуты, полнота, текущее описание, утверждения
`moderation://queue`	статический	заблокированные карточки и очередь по возрастанию уверенности

URI	Вид	Данные
rules://forbidden-phrases	статический	запрещённые формулировки
rules://required-attributes	статический	обязательные атрибуты
returns://policy/{category}	шаблон	условия возврата категории простым текстом
orders://{order_number}/summary	шаблон	статус, трек-номер, позиции, срок возврата; телефон в маске
audit://agent-log	статический	последние 50 действий агента

Шаблоны промптов: prepare_product_card (проверить атрибуты, запросить недостающие поля, не придумывая значений, вычитать черновик, сверить утверждения с атрибутами, отправить на модерацию после подтверждения), moderation_review (прочитать карточку и правила, составить отчёт о нарушениях, предложить решение) и return_request_assistant (сначала прочитать сводку заказа и правила возврата, затем проверить и оформить возврат). Все три требуют читать ресурсы до вызова изменяющих инструментов и выполнять необратимые действия только после подтверждения пользователя.

5.4 Реализация

Каждый инструмент выполняет бизнес-функцию через обёртку run(). Она записывает действие в журнал с результатом и превращает нарушение бизнес-правила (DomainError) в ToolError: клиент получает ответ с isError: true и текстом, который модель может пересказать пользователю.

```
    def run(action: str, entity: str, entity_id: str | None, payload: dict, fn: Callable[[], Any])
-> Any:
        """Выполняет действие агента и записывает его в журнал (успех или отказ)."""
        try:
            result = fn()
        except DomainError as exc:
            db.audit(conn, action, entity, entity_id, payload, "rejected")
            log.info("отклонено %s(%s): %s", action, entity_id, exc)
            raise ToolError(str(exc)) from exc
        db.audit(conn, action, entity, entity_id or _id_from(result), payload, "ok")
        return result
```

```
    @mcp.tool(annotations=WRITE)
    def create_return_request(
        order_number: OrderNumber,
        reason: Annotated[Literal["size_mismatch", "not_as_described", "defect", "changed_mind"],
                          Field(description="Причина возврата")],
        is_exchange: Annotated[bool, Field(description="true — обмен на другой размер, false —
возврат денег")] = False,
        order_item_id: Annotated[str | None, Field(description="UUID позиции заказа")] = None,
    ) -> dict[str, Any]:
        """Оформить возврат или обмен (UC-08). Нельзя для неполученного заказа, после истечения
срока
        и для категорий, не подлежащих возврату. Ответ содержит пункт приёма и срок сдачи."""
        return run("return.create", "return_request", order_number, {"reason": reason,
"is_exchange": is_exchange},
                   lambda: service.create_return(conn, order_number, order_item_id, reason,
is_exchange))
```

Проверка возможности возврата формирует понятную причину отказа, а не код ошибки (US-15):

```
def return_eligibility(conn: sqlite3.Connection, order_number: str, order_item_id: str | None,
                       today: date | None = None) -> dict:
    """US-11, US-15: доступен ли возврат и, если нет, понятная причина."""
    today = today or date.today()
    order, item = _order_with_item(conn, order_number, order_item_id)
    policy = conn.execute("SELECT * FROM return_policy WHERE category_id = ?",
(item["category_id"],)).fetchone()
    base = {"order_number": order_number, "order_item_id": item["id"], "title": item["title"]}
    if order["status"] != "delivered":
        return {**base, "eligible": False,
                "reason": f"Заказ ещё не получен (статус: {order['status']}). Возврат оформляется
после вручения; "
                          f"до передачи в доставку заказ можно отменить."}
    if policy is None or policy["return_forbidden"]:
        return {**base, "eligible": False,
                "reason": f"Товары категории «{item['category']}» надлежащего качества не подлежат
возврату "
                          f"по законодательству. При браке оформите заявку с причиной defect."}
    due = date.fromisoformat(order["return_due_date"])
    if today > due:
        return {**base, "eligible": False,
                "reason": f"Срок возврата истёк {due.strftime('%d.%m.%Y')} ({policy['period_days']}
дней с "
                          f"момента получения). При браке обратитесь к продавцу."}
    if conn.execute("SELECT 1 FROM return_request WHERE order_item_id = ?",
(item["id"],)).fetchone():
        return {**base, "eligible": False, "reason": "По этой позиции заявка на возврат уже
оформлена"}
    return {**base, "eligible": True, "return_due_date": due.isoformat(),
            "days_left": (due - today).days, "dropoff_point": order["delivery_address"]}
```

Проверка запрещённых формулировок выполняется на сервере детерминированно, а не «просьбой» к модели, поэтому её нельзя обойти изменением промпта (ОГР-1).

5.5 Тестирование

Сервер проверен двумя способами.

Автотесты (`pytest`, 23 теста) подключаются к серверу MCP-клиентом SDK в том же процессе; один тест запускает сервер как дочерний процесс и работает через настоящий stdio-транспорт. Результат: 23 из 23 пройдены (`mcp-server/results/pytest.log`).

Таблица 16 — Состав автотестов MCP-сервера

Группа	Что проверяется
Примитивы	состав 13 инструментов, 5 ресурсов, 3 шаблонов ресурсов, 3 промптов; типизация JSON-схем; аннотации `read_only`/`destructive`; отклонение аргумента `price = -5` по схеме
CRUD и описания	список недостающих полей; автоматический черновик после заполнения атрибутов, все утверждения с атрибутом-источником, summary ≤ 90; отказ генерации при неполных атрибутах; в черновик не попадает незаполненный «Уход»; блокировка «лучший» и запрет отправки; подтверждение удаления, защита товара из заказов

Группа	Что проверяется
Модерация	заблокированные отдельным блоком, очередь по возрастанию уверенности; обязательный комментарий; публикация; повторное решение по закрытой задаче отклоняется; новое правило сразу запрещает одобрение
Возвраты	успешный обмен с пунктом приёма и сроком; повторная заявка отклоняется; отказ с понятной причиной: истёк срок, заказ не получен, невозвратная категория, неизвестный заказ
Ресурсы и журнал	маскирование телефона; правила возврата; ошибка для несуществующего товара; запись действий с результатом без персональных данных; промпт требует прочитать ресурс до вызова инструмента
Транспорт	список инструментов и вызов через stdio

MCP Inspector — официальный инструмент отладки MCP — запущен в режиме CLI сценарием `mcp-server/inspector_check.sh` из 14 шагов, воспроизводящим диалог агента: список примитивов, чтение очереди модерации, отказ генерации при неполных атрибутах, заполнение атрибутов с автоматическим черновиком, отправка на модерацию, чтение сводки заказа, отказ в возврате после истечения срока, оформление обмена, получение промпта и чтение журнала. Все шаги выполнены; два запланированных отказа вернули `isError: true` с понятной причиной. Фрагмент лога:

```
=== 11. Возврат после истечения срока (ожидается отказ)
$ mcp-inspector --cli ... --method tools/call --tool-name create_return_request
      --tool-arg order_number=MS-2026-000101 reason=changed_mind
{"content": [{"type": "text", "text": "Error executing tool create_return_request: Возврат
недоступен:
   Срок возврата истёк 06.10.2026 (14 дней с момента получения). При браке обратитесь к
продавцу."}],
 "isError": true}

=== 12. Обмен по свежему заказу
"structuredContent": {"return_id": "cf7f3600-8c02-4666-8e98-cbe44329ed4e", "status":
"awaiting_dropoff",
   "is_exchange": true, "dropoff_point": "ПВЗ Европочта, Минск, пр. Независимости, 58",
   "dropoff_deadline": "2026-10-29", "photos_required": false}

=== 14. Журнал действий агента
{"action": "return.create", "entity_id": "MS-2026-000154", "payload": {"reason": "size_mismatch",
   "is_exchange": true}, "outcome": "ok"}
{"action": "return.create", "entity_id": "MS-2026-000101", "payload": {"reason": "changed_mind",
   "is_exchange": false}, "outcome": "rejected"}
```

Этот же сценарий в виде диалога пользователя с ассистентом показан на рисунке 8.

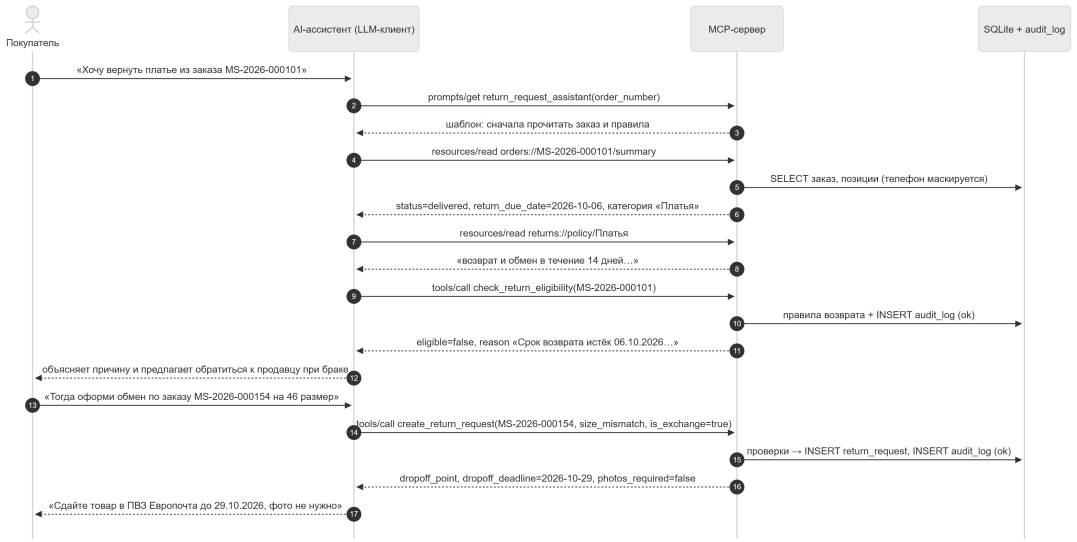


Рисунок 8 — Последовательность сценария возврата через AI-ассистента

5.6 Документация MCP-сервера

Файл `mcp-server/README.md` содержит инструкцию по установке и запуску, конфигурацию подключения к Claude Desktop и Cursor, команды проверки через MCP Inspector и pytest, описание сущностей, Tools, Resources и Prompts, правила безопасности и примеры запросов от лица продавца, модератора и покупателя с ожидаемой реакцией ассистента. Детали проектирования вынесены на страницу вики «MCP-сервер».

6 Документирование проекта

Документация подготовлена в двух формах с общим источником.

- **Вики** (`wiki/`): Главная, Функциональные требования, Диаграмма файлов, Дополнительная спецификация, Схема базы данных, API проекта, MCP-сервер и боковая навигация `_Sidebar.md`. Страницы готовы к загрузке в вики-репозиторий GitHub.
- **GitHub Pages** (`docs/`): тот же набор страниц с навигацией и темой Jekyll, собирается скриптом `tools/build_pages.py`; в `docs/api/` — Swagger UI со спецификацией. Сайт публикуется из каталога `docs` ветки `main`.
- **README** основного репозитория оформлен по разделам Project Name, Description, Installation, Sub modules, Usage, Contributing.

Связи между источниками и сгенерированными файлами показаны на диаграмме файлов (приложение Б). Пересборка документации после изменения требований, схемы или спецификации выполняется тремя командами и сопровождается проверкой согласованности.

Заключение

1. Требования ЛР№3 проанализированы с точки зрения API: найдены варианты использования без обосновывающих историй, операции, без которых API неполон, и истории, требующие отдельного варианта использования. Модель требований расширена до 32 User Story и 30 вариантов использования с сохранением нумерации ЛР № 3; каждый вариант использования реализован операцией REST API или инструментом MCP, что подтверждено автоматической проверкой.
2. Логическая модель ЛР № 3 уточнена (24 → 31 таблица): добавлены учётные записи, справочники, на которые ссылались внешние ключи, журналы нарушений и действий. Физическая модель для PostgreSQL 16 оформлена SQL-файлом, логической схемой и ER-диаграммами, построенными из этого файла. Проверка в реальной СУБД подтвердила, что ограничения отклоняют все девять видов некорректных данных, и выявила ошибку в `CHECK`-ограничении с `NULL`, которая была исправлена.
3. Спроектированы 45 маршрутов по 23 сущностям; 55 операций сопоставлены вариантам использования с указанием действующего лица и требования аутентификации.
4. Спецификация OpenAPI 3.0.3 проходит два валидатора без замечаний и определяет тела всех POST/PUT-запросов и смысл каждого кода ответа. Коллекция Bruno из 43 запросов (13 негативных) пройдена против mock-сервера Prism полностью — 65 тестов из 65.
5. Реализован MVP MCP-сервера на MCP Python SDK: 13 инструментов, 8 ресурсов, 3 шаблона промптов, mock-база SQLite. Бизнес-правила проверяются на сервере, действия агента журналируются без персональных данных. Сервер проверен 23 автотестами и сценарием официального MCP Inspector.
6. Подготовлены вики, сайт GitHub Pages со Swagger UI и README. Все таблицы документации генерируются из исходных файлов, поэтому изменения требований, схемы или спецификации переносятся в документацию автоматически.

Список использованных источников

1. Давидовская, М. И. Стили интеграции программных систем и проектирование API : методические указания по выполнению лабораторной работы № 4 / М. И. Давидовская. — Минск : БГУ, 2026. — 16 с.
2. Харченко, А. Лабораторная работа № 3. Разработка требований к программной системе / А. Харченко, В. Шепко, А. Степук. — Минск : БГУ, 2026.
3. GetAnalyst. Проектирование REST API на примере интернет-магазина [Электронный ресурс]. — Режим доступа: https://getanalyst.ru. — Дата доступа: 20.10.2026.
4. OpenAPI Specification. Version 3.0.3 [Электронный ресурс]. — Режим доступа: https://spec.openapis.org/oas/v3.0.3. — Дата доступа: 20.10.2026.
5. Swagger Petstore — OpenAPI 3.0 [Электронный ресурс]. — Режим доступа: https://petstore3.swagger.io. — Дата доступа: 20.10.2026.
6. RFC 9457. Problem Details for HTTP APIs [Электронный ресурс]. — Режим доступа: https://www.rfc-editor.org/rfc/rfc9457. — Дата доступа: 20.10.2026.
7. Bruno. API Documentation [Электронный ресурс]. — Режим доступа: https://docs.usebruno.com. — Дата доступа: 21.10.2026.
8. Stoplight Prism. Mock Servers [Электронный ресурс]. — Режим доступа: https://docs.stoplight.io/docs/prism. — Дата доступа: 21.10.2026.
9. Model Context Protocol. Specification [Электронный ресурс]. — Режим доступа: https://modelcontextprotocol.io/specification. — Дата доступа: 21.10.2026.
10. MCP Python SDK [Электронный ресурс]. — Режим доступа: https://github.com/modelcontextprotocol/python-sdk. — Дата доступа: 21.10.2026.
11. MCP Inspector [Электронный ресурс]. — Режим доступа: https://github.com/modelcontextprotocol/inspector. — Дата доступа: 22.10.2026.
12. PostgreSQL 16 Documentation [Электронный ресурс]. — Режим доступа: https://www.postgresql.org/docs/16/. — Дата доступа: 19.10.2026.

Приложение А. Логическая схема базы данных

Домен «Пользователи и доступ»

Сущность	Таблица БД	Поле	Смысл поля	Тип данных
Город	`city`	`id`	уникальный идентификатор города PK	UUID
		`name`	название населённого пункта	VARCHAR(100)
		`region`	область	VARCHAR(100)
Учётная запись	`app_user`	`id`	уникальный идентификатор учётной записи PK	UUID
		`login`	логин (адрес электронной почты) UK	VARCHAR(254)
		`password_hash`	хеш пароля (bcrypt)	VARCHAR(255)
		`role`	роль: покупатель, продавец, модератор, администратор	ENUM(buyer, seller, moderator, admin)
		`is_active`	признак активной учётной записи	BOOLEAN
		`created_at`	дата и время регистрации	TIMESTAMP
Токен обновления	`refresh_token`	`id`	уникальный идентификатор токена PK	UUID
		`user_id`	учётная запись-владелец FK → app_user	UUID
		`token_hash`	SHA-256 хеш токена (сам токен не хранится) UK	CHAR(64)
		`expires_at`	срок действия	TIMESTAMP
		`revoked_at`	момент отзыва (выход из системы) NULL	TIMESTAMP
Покупатель	`customer`	`id`	уникальный идентификатор покупателя PK	UUID
		`user_id`	учётная запись покупателя FK → app_user, UK	UUID
		`phone`	номер телефона в формате +375XXXXXXXXX UK, NULL	VARCHAR(16)
		`name`	имя для обращения NULL	VARCHAR(100)
		`city_id`	город покупателя FK → city, NULL	UUID
		`deleted_at`	момент обезличивания по запросу (ФТ-13.5) NULL	TIMESTAMP
Параметры размера	`size_params`	`customer_id`	покупатель PK, FK → customer	UUID
		`height_cm`	рост, см NULL	INTEGER
		`chest_cm`	обхват груди, см NULL	INTEGER
		`foot_length_cm`	длина стопы, см NULL	DECIMAL(4,1)

Сущность	Таблица БД	Поле	Смысл поля	Тип данных
		updated_at	дата последнего изменения	TIMESTAMP

Домен «Каталог»

Сущность	Таблица БД	Поле	Смысл поля	Тип данных
Продавец	seller	id	уникальный идентификатор продавца PK	UUID
		user_id	учётная запись продавца FK → app_user, UK	UUID
		brand_name	торговое наименование	VARCHAR(120)
		unp	учётный номер плательщика (9 цифр) UK	CHAR(9)
		rating	рейтинг качества карточек (US-19)	DECIMAL(3,2)
Бренд	brand	id	уникальный идентификатор бренда PK	UUID
		seller_id	продавец-владелец бренда FK → seller	UUID
		name	название бренда UK	VARCHAR(120)
		made_in_belarus	признак «Сделано в Беларуси» (US-13)	BOOLEAN
Категория	category	id	уникальный идентификатор категории PK	UUID
		parent_id	родительская категория FK → category, NULL	UUID
		name	название категории	VARCHAR(100)
		level	уровень вложенности (1 — корень)	INTEGER
Размерная сетка	size_chart	id	уникальный идентификатор сетки PK	UUID
		brand_id	бренд FK → brand	UUID
		item_type	тип изделия (кроссовки, платье и т. п.)	VARCHAR(60)
		fit_deviation	отклонение посадки в размерах (−0.5 — маломерит на полразмера)	DECIMAL(3,1)
Товар	product	id	уникальный идентификатор товара PK	UUID
		seller_id	продавец FK → seller	UUID
		category_id	категория каталога FK → category	UUID
		brand_id	бренд FK → brand	UUID
		size_chart_id	размерная сетка (0..1) FK → size_chart, NULL	UUID

Сущность	Таблица БД	Поле	Смысл поля	Тип данных
		sku	артикул продавца	VARCHAR(64)
		title	название товара	VARCHAR(200)
		price	цена, BYN	DECIMAL(10,2)
		season	сезон NULL	ENUM(summer, winter, demi, all_season)
		created_at	дата создания	TIMESTAMP
		updated_at	дата последнего изменения	TIMESTAMP
Вариант товара	product_variant	id	уникальный идентификатор варианта PK	UUID
		product_id	товар FK → product	UUID
		size	размер	VARCHAR(16)
		color	цвет	VARCHAR(40)
		barcode	штрихкод EAN-13 (поиск по штрихкоду, US-14) UK, NULL	VARCHAR(14)
Склад	warehouse	id	уникальный идентификатор склада PK	UUID
		name	название склада	VARCHAR(100)
		city_id	город расположения FK → city	UUID
Остаток	stock	variant_id	вариант товара PK, FK → product_variant	UUID
		warehouse_id	склад PK, FK → warehouse	UUID
		quantity	общее количество единиц	INTEGER
		reserved	зарезервировано под заказы	INTEGER
Справочник атрибутов	attribute_dict	code	код атрибута (material, season, country…) PK	VARCHAR(64)
		display_name	название поля в форме продавца	VARCHAR(100)
		required	обязателен для генерации описания	BOOLEAN
		sort_order	порядок вывода в форме	INTEGER
Атрибут товара	product_attribute	product_id	товар PK, FK → product	UUID
		code	код атрибута PK, FK → attribute_dict	VARCHAR(64)
		value	значение атрибута	TEXT

Домен «Описания и модерация»

Сущность	Таблица БД	Поле	Смысл поля	Тип данных
Описание товара	`description`	`id`	уникальный идентификатор описания PK	UUID
		`product_id`	товар (не более одного описания) FK → product, UK	UUID
		`status`	статус модерации	ENUM(incomplete, draft, pending_moderation, blocked, returned, published)
		`confidence`	уровень уверенности генератора NULL	ENUM(low, medium, high)
		`block_reason`	найденная запрещённая формулировка (US-09) NULL	TEXT
		`current_version`	номер актуальной версии	INTEGER
		`updated_at`	дата последнего изменения	TIMESTAMP
Версия описания	`description_version`	`id`	уникальный идентификатор версии PK	UUID
		`description_id`	описание FK → description	UUID
		`number`	порядковый номер версии	INTEGER
		`kind`	тип: полная версия или summary (AC-05.5)	ENUM(full, summary)
		`body`	текст версии	TEXT
		`author`	автор: generator или идентификатор продавца	VARCHAR(64)
		`prompt_version`	версия шаблона промпта (US-17) NULL	VARCHAR(16)
		`created_at`	дата создания	TIMESTAMP
Оценка достоверности	`description_claim`	`id`	уникальный идентификатор утверждения PK	UUID
		`description_id`	описание FK → description	UUID
		`statement`	текст утверждения	TEXT
		`source_attribute`	атрибут-источник FK → attribute_dict, NULL	VARCHAR(64)
		`verified`	подтверждено атрибутом	BOOLEAN
Правило генерации	`generation_rule`	`id`	уникальный идентификатор правила PK	UUID

Сущность	Таблица БД	Поле	Смысл поля	Тип данных
		phrase	запрещённая формулировка UK	VARCHAR(200)
		is_active	правило действует	BOOLEAN
		created_by	администратор, создавший правило FK → app_user, NULL	UUID
		created_at	дата создания	TIMESTAMP
Задача модерации	moderation_task	id	уникальный идентификатор задачи PK	UUID
		description_id	проверяемое описание FK → description	UUID
		moderator_id	модератор, взявший задачу в работу FK → app_user, NULL	UUID
		claimed_at	момент захвата задачи NULL	TIMESTAMP
		locked_until	окончание блокировки для других модераторов (АС-08.7) NULL	TIMESTAMP
		created_at	момент постановки в очередь	TIMESTAMP
		closed_at	момент закрытия решением NULL	TIMESTAMP
Решение модератора	moderation_decision	id	уникальный идентификатор решения PK	UUID
		task_id	задача модерации FK → moderation_task, UK	UUID
		verdict	вердикт: одобрить или вернуть продавцу	ENUM(approve, return)
		comment	комментарий (обязателен при возврате) NULL	TEXT
		duration_sec	длительность проверки, с (ФТ-12.3) NULL	INTEGER
		decided_at	момент решения	TIMESTAMP
Шаблон комментария	comment_template	id	уникальный идентификатор шаблона PK	UUID
		title	краткое название UK	VARCHAR(100)
		body	текст комментария	TEXT
		is_active	шаблон доступен модераторам	BOOLEAN
Нарушение продавца	seller_violation	id	уникальный идентификатор записи PK	UUID
		seller_id	продавец FK → seller	UUID

Сущность	Таблица БД	Поле	Смысл поля	Тип данных
		description_id	заблокированное описание FK → description, NULL	UUID
		phrase	найденная формулировка	VARCHAR(200)
		created_at	момент блокировки	TIMESTAMP

Домен «Заказы, возвраты, отзывы»

Сущность	Таблица БД	Поле	Смысл поля	Тип данных
Заказ	orders	id	уникальный идентификатор заказа PK	UUID
		customer_id	покупатель FK → customer	UUID
		number	номер заказа для покупателя UK	VARCHAR(20)
		status	статус заказа	ENUM(created, paid, assembling, shipped, delivered, cancelled)
		payment_method	способ оплаты	ENUM(card, erip, cash_on_delivery)
		delivery_method	способ доставки	ENUM(pickup_point, courier, post)
		delivery_address	адрес или пункт выдачи	TEXT
		total	сумма заказа, BYN	DECIMAL(10,2)
		created_at	дата оформления	TIMESTAMP
		delivered_at	дата вручения NULL	TIMESTAMP
		return_due_date	последний день возврата NULL	DATE
Позиция заказа	order_item	id	уникальный идентификатор позиции PK	UUID
		order_id	заказ FK → orders	UUID
		variant_id	вариант товара FK → product_variant	UUID
		quantity	количество	INTEGER
		price_at_order	цена на момент заказа, BYN	DECIMAL(10,2)
Платёж	payment	id	уникальный идентификатор платежа PK	UUID

Сущность	Таблица БД	Поле	Смысл поля	Тип данных
		order_id	заказ FK → orders	UUID
		amount	сумма, BYN	DECIMAL(10,2)
		external_id	идентификатор в платёжной системе (идемпотентность вебхука, ФТ-15.6) UK, NULL	VARCHAR(64)
		status	статус платежа	ENUM(pending, succeeded, failed, refunded)
		updated_at	дата последнего изменения статуса	TIMESTAMP
Правила возврата	return_policy	id	уникальный идентификатор правила PK	UUID
		category_id	категория FK → category, UK	UUID
		period_days	срок возврата, дней	INTEGER
		return_forbidden	возврат запрещён законодательством (AC-12.4)	BOOLEAN
Заявка на возврат	return_request	id	уникальный идентификатор заявки PK	UUID
		order_id	заказ FK → orders	UUID
		order_item_id	возвращаемая позиция (частичный возврат) FK → order_item	UUID
		reason	причина возврата	ENUM(size_mismatch, not_as_described, defect, changed_mind)
		status	статус заявки	ENUM(created, awaiting_dropoff, received, approved, rejected, refunded)
		is_exchange	обмен вместо возврата денег	BOOLEAN
		comment	комментарий покупателя NULL	TEXT
		dropoff_point	пункт приёма (AC-11.5) NULL	VARCHAR(200)
		created_at	дата создания	TIMESTAMP
Фото к заявке	return_photo	id	уникальный идентификатор фото PK	UUID
		return_request_id	заявка FK → return_request	UUID
		url	ссылка на объект в S3	VARCHAR(500)

Сущность	Таблица БД	Поле	Смысл поля	Тип данных
Отзыв	`review`	`id`	уникальный идентификатор отзыва PK	UUID
		`product_id`	товар FK → product	UUID
		`customer_id`	покупатель FK → customer	UUID
		`rating`	оценка от 1 до 5	INTEGER
		`fit_rating`	оценка посадки NULL	ENUM(runs_small, true_to_size, runs_large)
		`body`	текст отзыва NULL	TEXT
		`created_at`	дата публикации	TIMESTAMP

Домен «Пользователи и доступ»

Сущность	Таблица БД	Поле	Смысл поля	Тип данных
Журнал действий	`audit_log`	`id`	порядковый номер записи PK	BIGINT
		`actor_kind`	тип инициатора: пользователь, MCP-агент, система	ENUM(user, mcp_agent, system)
		`actor_id`	учётная запись инициатора FK → app_user, NULL	UUID
		`action`	выполненное действие	VARCHAR(64)
		`entity`	тип затронутой сущности	VARCHAR(64)
		`entity_id`	идентификатор затронутой сущности NULL	VARCHAR(64)
		`payload`	параметры действия без персональных данных	JSON
		`created_at`	момент действия	TIMESTAMP

Приложение Б. Диаграмма файлов проекта

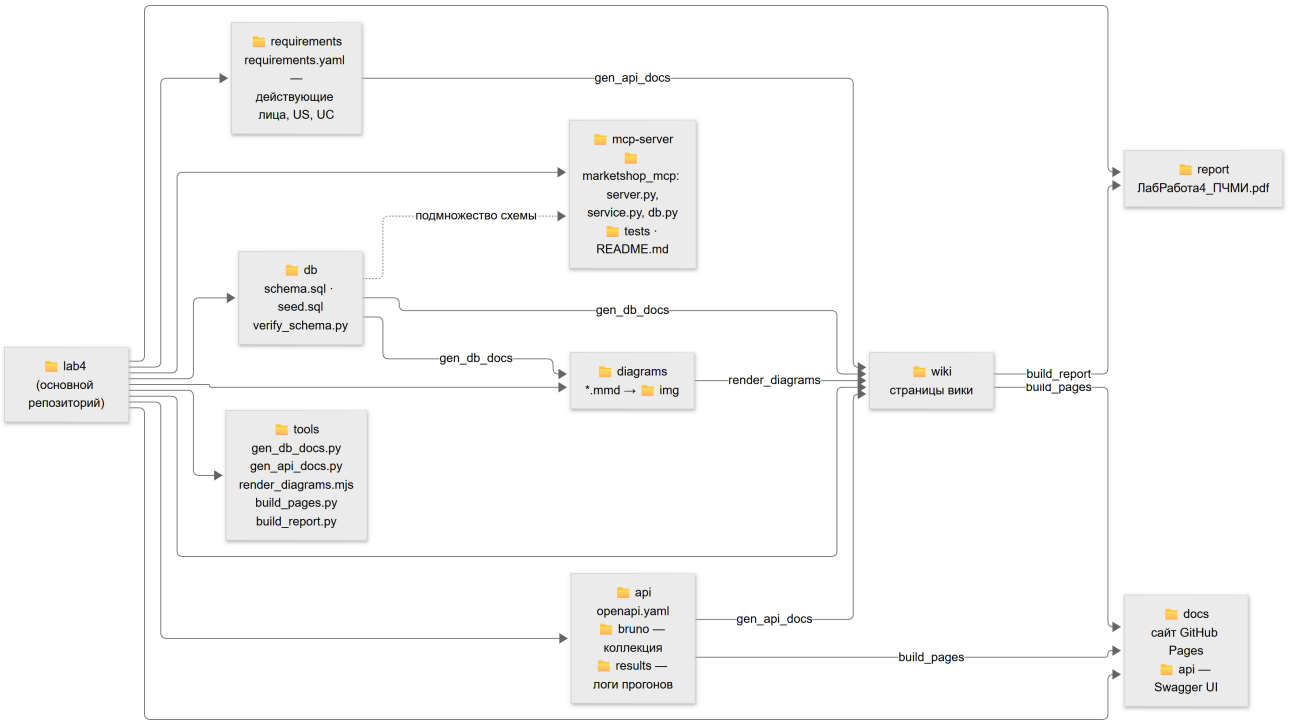


Рисунок 9 — Структура репозитория и порядок генерации артефактов